

Instituto Politécnico Nacional

Escuela Superior de Cómputo



Análisis y Diseño de Algoritmos

Práctica:

Programación Dinámica II.

Fecha: 11/12/25

Domínguez Jiménez Ana Andrea

1. Ejercicios de programación *(Lenguaje C)*

a) Pseudocódigo 1 (Recursivo):

Escribir las formas distintas de escribir la suma de ***n*** como la suma de ***1, 3,*** y ***4.***

Input: int n

Output: int F->Formas en las que se puede sumar esa n

Suma_R(int n):

//Caso base:

if(n==0) return *1*; //->suma cero, solo *una* forma posible

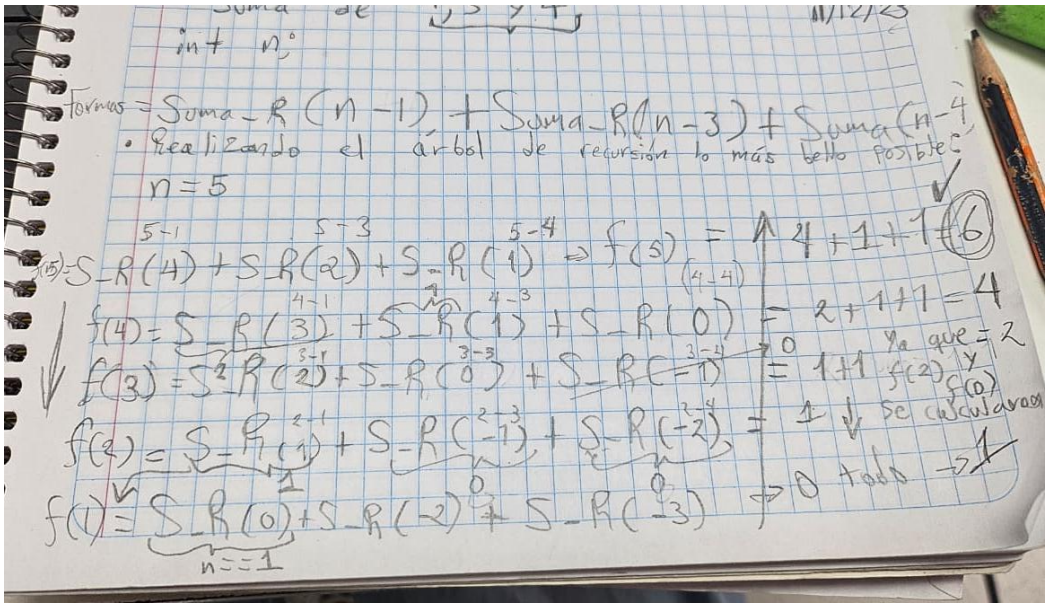
if(n<0) return 0; //-> No es posible

if(n>0) then

return Suma_R(n-1)+Suma_R(n-3)+Suma_R(n-4)

Como es recursivo va a depender de n, entonces su complejidad es de **O(n)**

Para mejor representación, realice el árbol de recursión:



En donde se hace la función de forma recursiva de:

Suma_R(n-1) +Suma_R(n-3) +Suma(n-4)

Me pareció muy similar a los algoritmos de factorial y Fibonacci, combinado con la lógica del cambio de moneda, ya que utiliza recursión y almacena el valor calculado para cada n para reutilizarlo posteriormente.

b) Pseudocódigo (PD):

Input: int n

Output: int F

Suma_PD(int n)

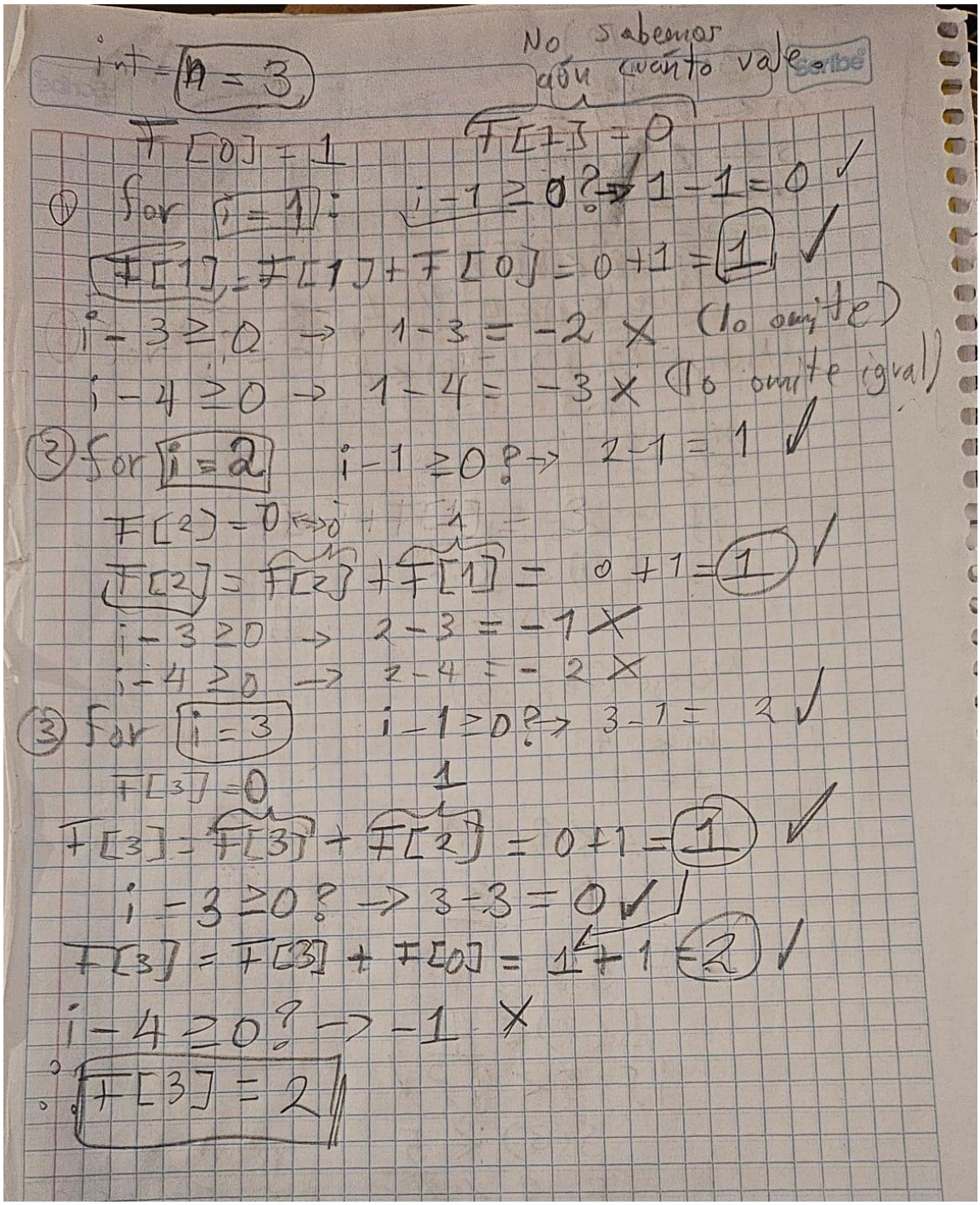
int F[] -> //Se guardan las diferentes formas

```
//Caso base
F[0]=1
if(n>0) then
  for i=1; to n do
  F[i]=0 -> inicializamos
if i-1 >= 0 then
F[i]= F[i]+F[i-1];
endif
if i-3 >= 0 then
F[i]= F[i] + F[i-3];
endif
if i-4 >= 0 then
F[i]= F[i] + F[i-3];
endif
  endfor
endif
return F[n]
```

Esto tiene complejidad de O(n)

Ejemplo:

Para ***n*=3.**



En cada iteración busca el valor de cada ***F[i]*** utilizando los resultados anteriores.

c)

```c
//Sesión 10. Diferentes formas de sumar un número.
#include<stdio.h>
int Suma_PD(int n);
int main(){
    int n, formas;

    printf("Ingrese un numero entero: ");
    scanf("%d", &n);

    formas= Suma_PD(n);
    printf("Hay %d formas diferentes de sumar %d", formas, n);

    return 0;
}
int Suma_PD(int n){
    int F[n+1];
    //Caso base:
    F[0]=1;

    if(n>0){
        for(int i=1; i<=n;i++){
            F[i]=0;
            if(i-1>=0){
                F[i]+=F[i-1];
            }
            if(i-3>=0){
                F[i]+=F[i-3];
            }
            if(i-4>=0){
                F[i]+=F[i-4];
            }
        }
    }

    return F[n];
}
```

2. Implementación de la mochila entera.

Lo primero que hice fue crear un archivo "*mochila1.txt*" del cual fue estructurado de la siguiente manera:

```
n W
peso1 valor1
peso2 valor2
...
pesowi valorvi
```

<-Orden

```
3 5
1 3
3 4
4 5
```

<- mochila1.txt

Se ordena así para que el programa lo entienda y lea bien.

```c
//Mochila entera
#include<stdio.h>
int main(){
    FILE *arch;
    FILE *matriz;
    char nombre[30];
    int n,W;

    printf("Ingrese el nombre del archivo: ");
    scanf("%s", nombre);

    arch=fopen(nombre, "r");
    if(arch==NULL){
        printf("\nEl archivo no existe.\n");
    }
    //leer numero de objetos y la capacidad
    fscanf(arch, "%d %d", &n, &W);
    int wi[n+1], vi[n+1];
    int V[n+1][W+1];

    for(int i=1;i<=n;i++){
        fscanf(arch, "%d %d", &wi[i], &vi[i]);//lee
    }
    printf("\nDatos leidos del archivo:\n");
    printf("Objetos: %d\n", n);
    printf("Capacidad: %d\n", W);

    for(int i = 1; i <= n; i++){
        printf("Objeto %d -> Peso: %d  Valor: %d\n", i, wi[i], vi[i]);
    }

    for(int i=0; i<=n;i++){
        for(int w=0; w<=W;w++){
            if(i==0 || w==0){
                V[i][w]=0;

            }else if(wi[i]<=w){//max

                int si=vi[i]+V[i-1][w-wi[i]];
                int no= V[i-1][w];

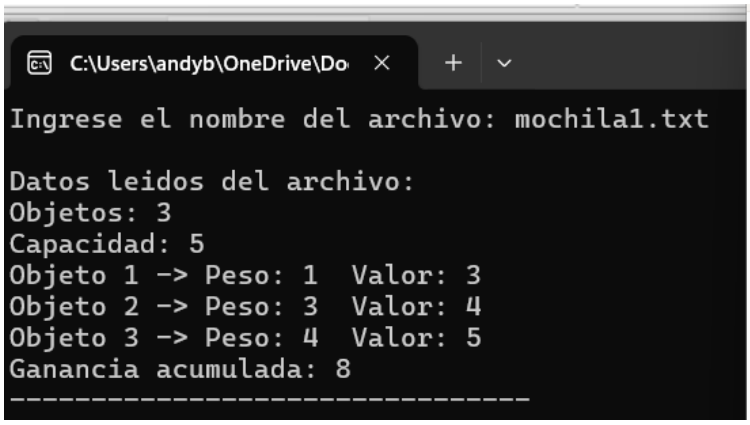
                if(si>no){
                    V[i][w]=si;
                }else{
                    V[i][w]=no;
                }

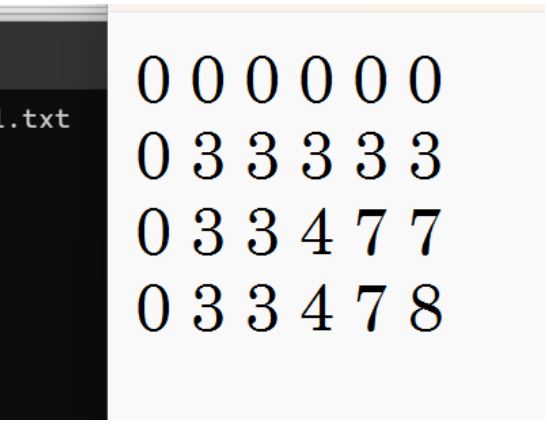
            }else{
                V[i][w]= V[i-1][w];//no lo toma
            }
        }
    }
    printf("Ganancia acumulada: %d", V[n][W]);
    fclose(arch);

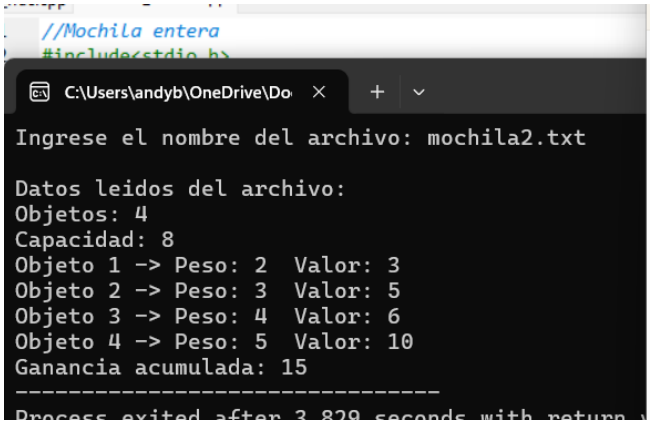

    matriz = fopen("matriz.txt", "w");

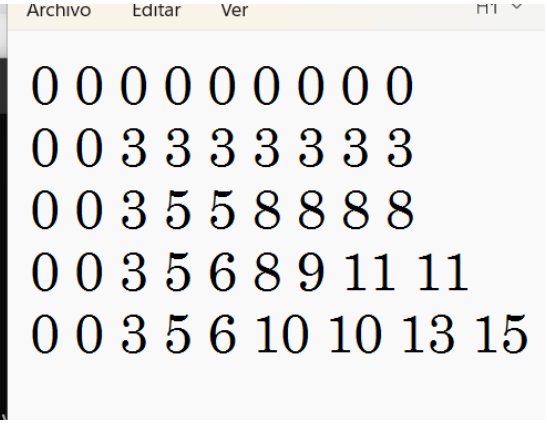
    for(int i = 0; i <= n; i++){
        for(int w = 0; w <= W; w++){
            fprintf(matriz, "%d ", V[i][w]);
        }
        fprintf(matriz, "\n");
    }
    fclose(matriz);
    return 0;
}
```

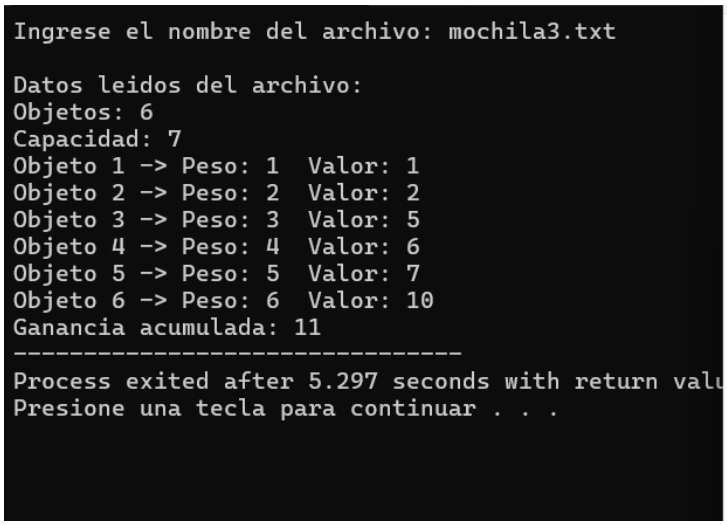
Lee los datos del archivo y después realiza la matriz de ganancia máxima *V[n,W]* y la escribe en un archivo, ejemplo de los 5 archivos mochila:

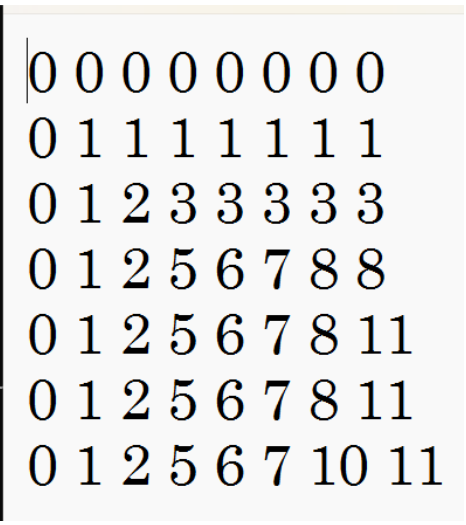


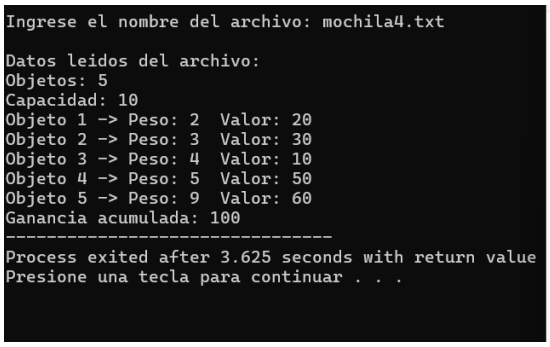




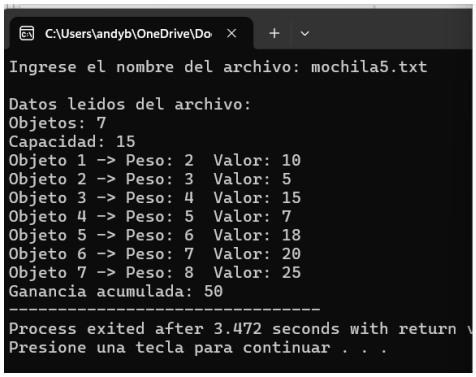








0 0 0 0 0 0 0 0 0 0 0
0 0 20 20 20 20 20 20 20 20 20
0 0 20 30 30 50 50 50 50 50 50
0 0 20 30 30 50 50 50 50 60 60
0 0 20 30 30 50 50 70 80 80 100
0 0 20 30 30 50 50 70 80 80 100



0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 10 10 10 10 10 10 10 10 10 10 10 10 10 10
0 0 10 10 10 15 15 15 15 15 15 15 15 15 15 15
0 0 10 10 15 15 25 25 25 30 30 30 30 30 30 30
0 0 10 10 15 15 25 25 25 30 30 32 32 32 37 37
0 0 10 10 15 15 25 25 28 30 33 33 43 43 43 48
0 0 10 10 15 15 25 25 28 30 33 35 43 45 45 48
0 0 10 10 15 15 25 25 28 30 35 35 43 45 50 50

3. Complementando lo anterior.

Modificando un poco el código anterior para que ahora el usuario indique el archivo de la matriz, para eso cree 5 documentos de texto diferentes para almacenar las matrices anteriores *"matriz1.txt" a "matriz5.txt"* y los 5 documentos *"mochila1.txt" a "mochila5.txt"* anteriores, para poder conectarlos cuando el usuario ingrese el nombre del archivo de la matriz, y así el programa pueda identificar los datos de los objetos *(vi, wi)* que le corresponde a cada matriz.

```c
//Matriz entera 2
#include<stdio.h>
#include<string.h>
int main(){
    FILE *matriz;
    FILE *mochila;
    char nombre[30], nom_mochila[30];

    printf("Ingrese el nombre del archivo: ");
    scanf("%s", nombre);

    matriz=fopen(nombre, "r");
    if(matriz==NULL){
        printf("\nEl archivo no existe.\n");
        return 1;//detiene todo
    }

    if(strcmp(nombre, "matriz1.txt")==0){
        strcpy(nom_mochila,"mochila1.txt");
    }else if(strcmp(nombre, "matriz2.txt")==0){
        strcpy(nom_mochila, "mochila2.txt");
    }else if(strcmp(nombre, "matriz3.txt")==0){
        strcpy(nom_mochila, "mochila3.txt");
    }else if(strcmp(nombre, "matriz4.txt")==0){
        strcpy(nom_mochila,"mochila4.txt");
    }else if(strcmp(nombre, "matriz5.txt")==0){
        strcpy(nom_mochila, "mochila5.txt");
    }

    mochila=fopen(nom_mochila, "r");
    if(mochila==NULL){
        printf("\nNo se pudo conectar el archivo.");
        return 1;
    }
```

Aquí hace la comparación para hacer la conexión entre archivos.

```
37      int colum=0, fila=0;
38      // lee primera línea
39      if (fgets(linea, sizeof(linea), matriz)) {
40          char *ptr = linea;
41          int valor;
42          while (sscanf(ptr, "%d", &valor) == 1) {//cuenta las columnas
43              colum++;
44              // recorre la linea completa ignorando los espacios
45              while (*ptr && *ptr != ' ' && *ptr != '\n') ptr++;
46              while (*ptr == ' ') ptr++;
47          }
48          fila = 1; //se conto la primera fila
49      }
50      // Contar filas restantes
51      while (fgets(linea, sizeof(linea), matriz)) {
52          fila++;
53      }
54      fclose(matriz);
55
56      int n= fila-1;
57      int W= colum-1;
58      int wi[n+1],vi[n+1];
59      int V[n+1][W+1];
60      printf("Objetos = %d\nCapacidad de la mochila= %d\n", n, W);
61
62      for(int i=1; i<=n;i++){
63          if(fscanf(mochila,"%d %d", &wi[i], &vi[i])!=2){//si no lee los dos datos
64          printf("\nArchivo vacio.");
65          fclose(mochila);
66          return 1;
67          }
68      }
69      //se copia lo mismo para hacer la matriz del anterior programa
```

Cuenta las filas y las columnas de la matriz seleccionada para determinar la capacidad de la mochila ***W*** y la cantidad de objetos ***n***.

```
    //se copia lo mismo para hacer la matriz del anterior programa
    for(int i=0; i<=n;i++){
        for(int w=0; w<=W;w++){//w peso actual
            if(i==0 || w==0){
                V[i][w]=0;

            }else if(wi[i]<=w){//si o no

                int si=vi[i]+V[i-1][w-wi[i]];
                int no= V[i-1][w];

                if(si>no){
                    V[i][w]=si;
                }else{
                    V[i][w]=no;
                }

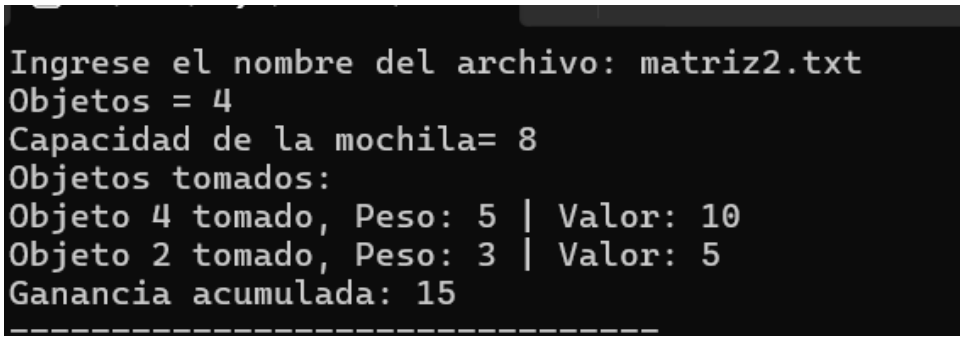
            }else{
                V[i][w]= V[i-1][w];//no lo toma
            }
        }
    }
    int w = W;
    printf("Objetos tomados:\n");
    for (int i = n; i > 0; i--) {
        if (V[i][w] != V[i-1][w]) {
            printf("Objeto %d tomado, Peso: %d | Valor: %d\n", i, wi[i], vi[i]);
            w -= wi[i];//capacidad actualizada
        }
    }//O(nW) depende de la cantidad de objetos y la capacidad de la mochila

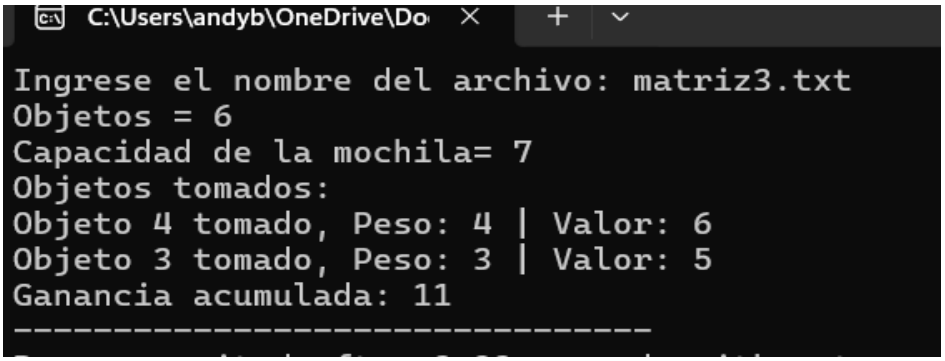
    printf("Ganancia acumulada: %d", V[n][W]);
    fclose(mochila);

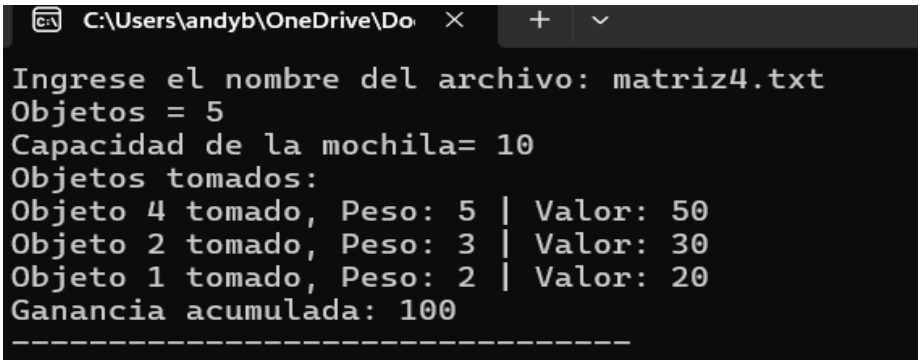
    return 0;
}
```

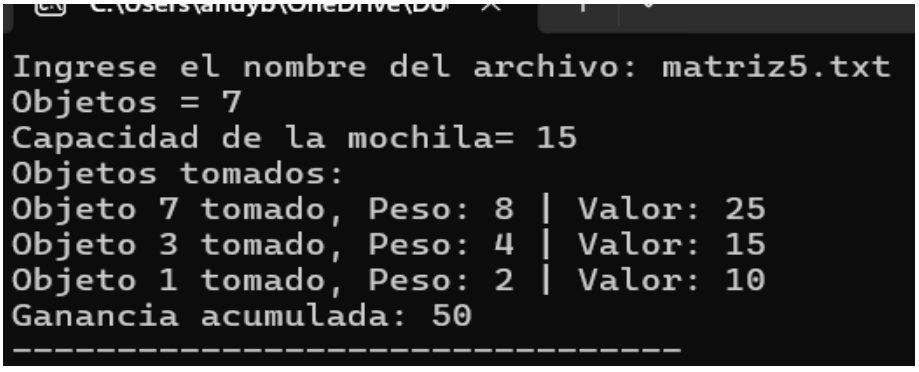
Se copia la matriz de ganancia máxima y cuenta los objetos tomados. Ejemplo con *matriz1.txt -> mochila1.txt*:

```
C:\Users\andyb\OneDrive\Do
Ingrese el nombre del archivo: matriz1.txt
Objetos = 3
Capacidad de la mochila= 5
Objetos tomados:
Objeto 3 tomado, Peso: 4 | Valor: 5
Objeto 1 tomado, Peso: 1 | Valor: 3
Ganancia acumulada: 8
--------------------------------
```









Como depende de la cantidad de objetos y la capacidad de la mochila su ***complejidad computacional es O(nW).***